

The MLFLOW logo is centered on a white background. It features the word "MLFLOW" in a bold, dark blue, sans-serif font. The background is decorated with abstract, overlapping geometric shapes in various shades of blue, including light blue, medium blue, and dark blue, creating a modern and dynamic feel.

MLFLOW

What and Why MLFLOW

- ▶ Various statistics say that between 50% and 90% of the models developed do not make it to production. This is often due to a failure to structure the work.
- ▶ Often the skills acquired in academia (or on Kaggle) are not sufficient to be able to put on a machine learning-based system that will be used by thousands of people.
- ▶ One of the most in-demand skills when looking for Machine Learning jobs in industry is the ability to use tools that enable the orchestration of complex pipelines such as MLflow.

- ▶ MLflow is an open-source platform designed to manage the end-to-end machine learning and AI lifecycle, focusing on **key components** like **Tracking**, **Projects**, and **Models**.
- ▶ It helps teams handle experiment **tracking, model packaging, deployment**, and generative AI evaluation

- ▶ [MLflow](#) is an open-source platform for end-to-end lifecycle management of Machine Learning developed by [Databricks](#).

Core Components

The background of the slide features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the right side and bottom of the frame, creating a modern, dynamic aesthetic.



mlflow

```
graph TD; MLflow[mlflow] --> Tracking[MLflow Tracking]; MLflow --> Projects[MLflow Projects]; MLflow --> Models[MLflow Models]; MLflow --> Registry[MLflow Registry]; Tracking --> TrackingDesc[Record and query experiments: code, data and results.]; Projects --> ProjectsDesc[Package data science code in a format to reproduce runs on any platform]; Models --> ModelsDesc[Deploy machine learning models in diverse serving environments]; Registry --> RegistryDesc[Store, annotate, discover and manage models in a central repository];
```

MLflow Tracking

Record and query experiments: code, data and results.

MLflow Projects

Package data science code in a format to reproduce runs on any platform

MLflow Models

Deploy machine learning models in diverse serving environments

MLflow Registry

Store, annotate, discover and manage models in a central repository

Tracking:

- ▶ Records parameters, code versions, metrics, and output files (artifacts) during training runs.
- ▶ Logs and organizes experiments, recording parameters, metrics, and artifacts.
- ▶ Compares different training runs to find the best model.
- ▶ Uses a backend store for metadata and an artifact store for output files

Projects:

- ▶ Packages machine learning code into a standard, reusable format for easy sharing and reproducibility.
- ▶ Standardizes code packaging for reproducibility and sharing.
- ▶ Organizes code and dependencies so data science code runs in any environment.

Models:

- ▶ Creates a standard format to manage and deploy models from any ML library to various hosting services.
- ▶ Provides a standard format for packaging machine learning models.
- ▶ Supports multiple frameworks (like PyTorch and Scikit-learn) for easy deployment

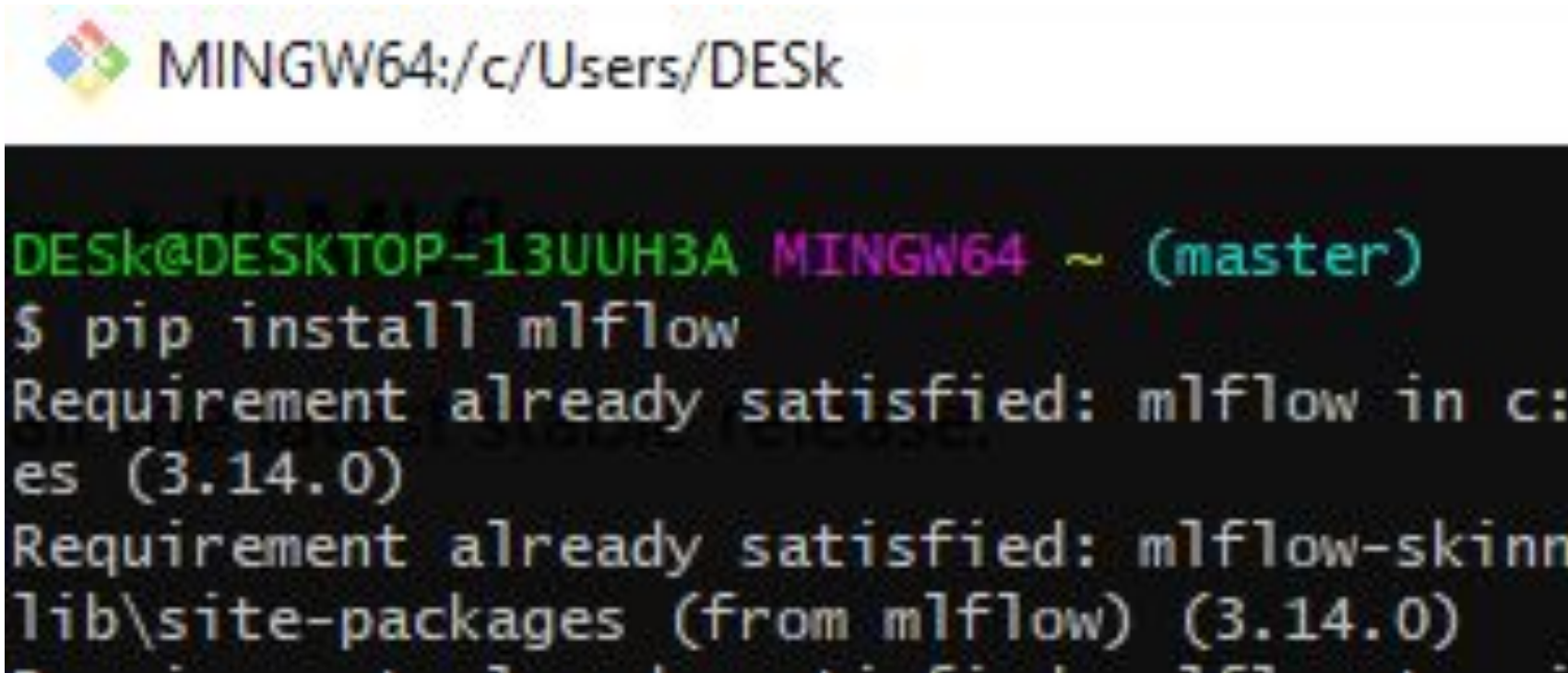
Model Registry:

- ▶ Acts as a centralized store for managing model versions, stage transitions (like staging to production), and annotations.
- ▶ Provides a standard format for packaging machine learning models.
- ▶ Supports multiple frameworks (like PyTorch and Scikit-learn) for easy deployment

Install MLflow

Open git bash and type the following

`pip install mlflow`

A screenshot of a Windows command prompt window. The title bar shows the Windows logo and the path 'MINGW64:/c/Users/DESk'. The terminal text shows the user 'DESk@DESKTOP-13UUH3A' in a 'MINGW64 ~ (master)' environment. They run the command '\$ pip install mlflow'. The output indicates that the requirement is already satisfied, showing 'mlflow in c:\es (3.14.0)' and 'mlflow-skinny in c:\lib\site-packages (from mlflow) (3.14.0)'.

```
MINGW64:/c/Users/DESk

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install mlflow
Requirement already satisfied: mlflow in c:\es (3.14.0)
Requirement already satisfied: mlflow-skinny in c:\lib\site-packages (from mlflow) (3.14.0)
```

Verify the installation

- ▶ Type the following
 - ▶ **mlflow --version**

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ mlflow --version
mlflow, version 3.14.0
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ python -c "import mlflow; print(mlflow.__version__)"
3.14.0
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

Install Common ML Libraries

- ▶ `pip install scikit-learn pandas numpy matplotlib seaborn`

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ ^C
Install Common ML Libraries
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install scikit-learn pandas numpy matplotlib seaborn
Requirement already satisfied: scikit-learn in c:\users\desk\anaco
packages (1.6.1)
Requirement already satisfied: pandas in c:\users\desk\anaconda3\T
es (2.2.3)
```


For deep learning

- ▶ `pip install tensorflow`
- ▶ Or
- ▶ `pip install torch torchvision`

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master) !otlib seaborn
$
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install tensorflow
Collecting tensorflow
  Downloading tensorflow-2.21.0-cp313-cp313-win_amd64.whl.metadata (4.5 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.5.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
```

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install torch torchvision
Requirement already satisfied: torch in c:\users\desk\anaconda3\lib\
s (2.9.0)
Collecting torchvision
  Downloading torchvision-0.28.0-cp313-cp313-win_amd64.whl.metadata
Requirement already satisfied: filelock in c:\users\desk\anaconda3\l
ages (from torch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users
da3\lib\site-packages (from torch) (4.12.2)
Requirement already satisfied: sympy<=1.13.3 in c:\users\desk\anacon
```

For gradient boosting

- ▶ `pip install xgboost lightgbm catboost`

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install torch torchvision
Requirement already satisfied: torch in c:\users\desk\anaconda3\lib\
s (2.9.0)
Collecting torchvision
  Downloading torchvision-0.28.0-cp313-cp313-win_amd64.whl.metadata
Requirement already satisfied: filelock in c:\users\desk\anaconda3\l
ages (from torch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users
da3\lib\site-packages (from torch) (4.12.2)
Requirement already satisfied: sympy>=1.13.3 in c:\users\desk\anacon
```

Start the MLflow UI

- ▶ If you're using the default local file backend then run the following
- ▶ `mlflow ui`
- ▶ It will generate local IP Address

Copy AND PASTE
THIS ADDRESS IN
BROUSER IT WILL
OPEN MLFLOW
WINDOW

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ mlflow ui
Backend store URI not provided. Using sqlite:///mlflow.db
Registry store URI not provided. Using backend store URI.
[MLflow] Security middleware enabled with default settings (localhost-only). To
allow connections from other hosts, use --host 0.0.0.0 and configure --allowed-h
osts and --cors-allowed-origins.
2026/07/28 20:28:54 WARNING mlflow.server: MLflow job execution requirements not
met (MLflow job backend does not support Windows system.). Server will start wi
thout job execution support. Errors will be surfaced at job invocation time.
2026/07/28 20:28:55 INFO:      Uvicorn running on http://127.0.0.1:5000 (Press CT
RL+C to quit)
2026/07/28 20:28:55 INFO:      Started parent process [12345]
```


Welcome to MLflow

Information about UI telemetry
MLflow collects usage data to improve the product. To confirm your data is collected, please visit the [documentation](#).

Getting Started



New to MLflow?
Explore MLflow's core features with

Tracing
Capture and debug LLM interactions and agent workflows.

Evaluation
Measure and compare LLM quality with built-in and

Recent Experiments

☐	Name	Time created
☐	MLflow Demo Experiment ⓘ	07/26/2026, 12:40:40 PM
☐	test	07/26/2026, 12:46:01 PM
☐		

EXPERIMENTS

mlflow

3.14.0

GenAI

Model training

Home

Experiments

Prompts

AI Gateway

New

Experiments

Search experiments

Q

Tag

	Name	Time created	Last modified	Description	Tags
☐	test	07/26/2026, 12:46:01 PM	07/26/2026, 12:46:01 PM	-	
☐	Simple_Linear_Regression	07/26/2026, 12:42:45 PM	07/26/2026, 12:42:45 PM	-	
☐	MLflow Demo Experiment	07/26/2026, 12:40:40 PM	07/26/2026, 12:40:40 PM	Sample experiment with pre-...	
☐	Default	07/26/2026, 12:38:08 PM	07/26/2026, 12:38:08 PM	-	

Create

Install Database Backend (Optional)

- ▶ Instead of storing runs in local files, you can use a database
- ▶ FOR MySQL
- ▶ `pip install pymysql`

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install pymysql
Collecting pymysql
  Downloading pymysql-1.2.0-py3-none-any.whl.metadata (4.3 kB)
  Downloading pymysql-1.2.0-py3-none-any.whl (45 kB)
Installing collected packages: pymysql
Successfully installed pymysql-1.2.0

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

Install Cloud Storage Support (Optional)

- ▶ FOR
- ▶ AWS S3 CLOUD

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ pip install boto3
Requirement already satisfied: boto3 in c:\users\desk\anaconda3\l...
s (1.43.56)
Requirement already satisfied: botocore<1.44.0,>=1.43.56 in c:\us...
da3\lib\site-packages (from boto3) (1.43.56)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in c:\users\
\lib\site-packages (from boto3) (1.0.1)
```

- ▶ Azure Blob Storage
- ▶ `pip install azure-storage-blob`

- ▶ Google Cloud Storage
- ▶ `pip install google-cloud-storage`

Install MLflow Extras

- ▶ Some useful packages:
- ▶ `pip install sqlalchemy`
- ▶ `pip install gunicorn # Linux deployment`
- ▶ `pip install waitress # Windows deployment`

Install Jupyter Support

- ▶ `pip install notebook ipykernel`

Test MLflow

- ▶ CREATE
- ▶ test_mlflow.py

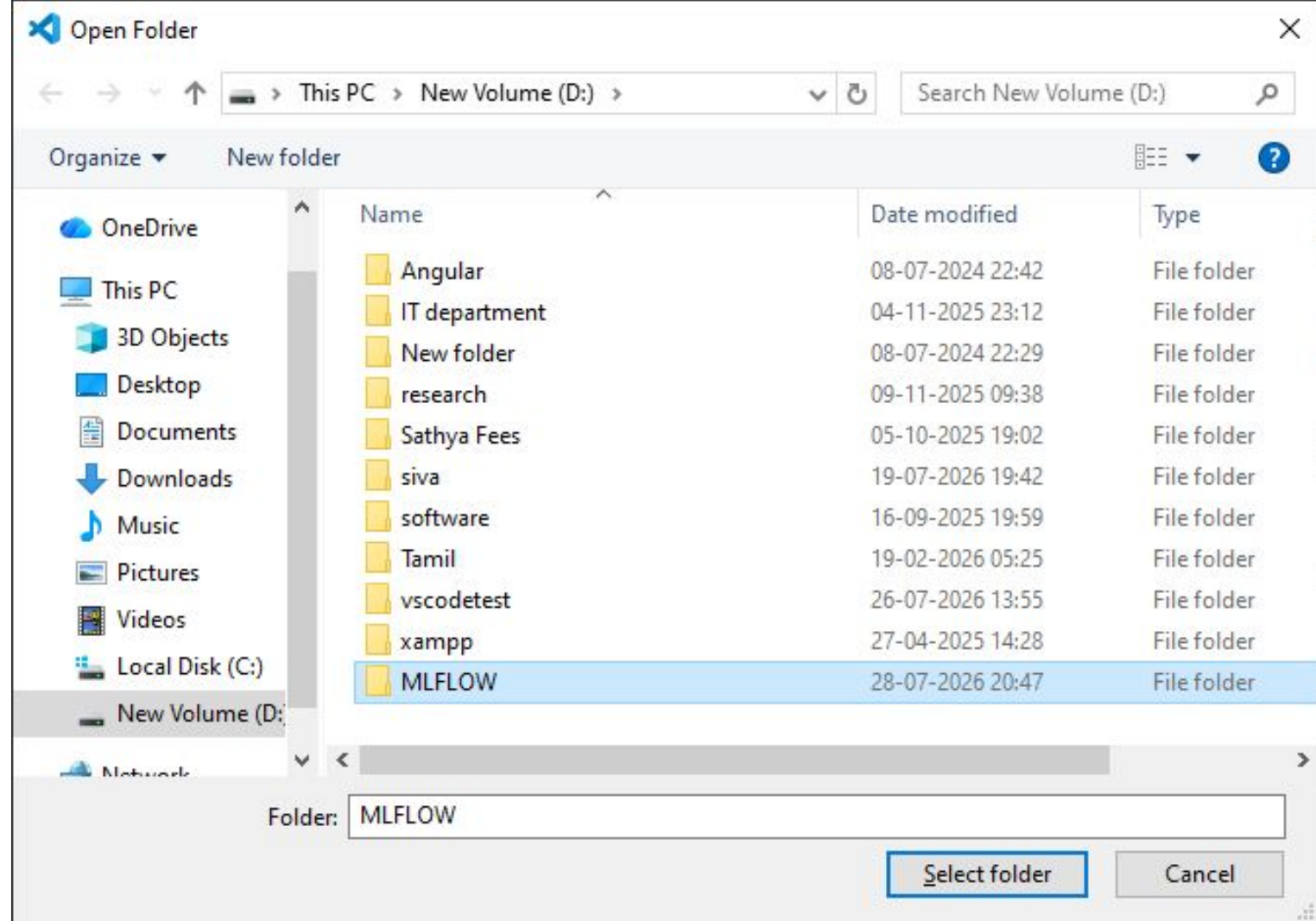
```
import mlflow

with mlflow.start_run():
    mlflow.log_param("learning_rate", 0.01)
    mlflow.log_metric("accuracy", 0.95)

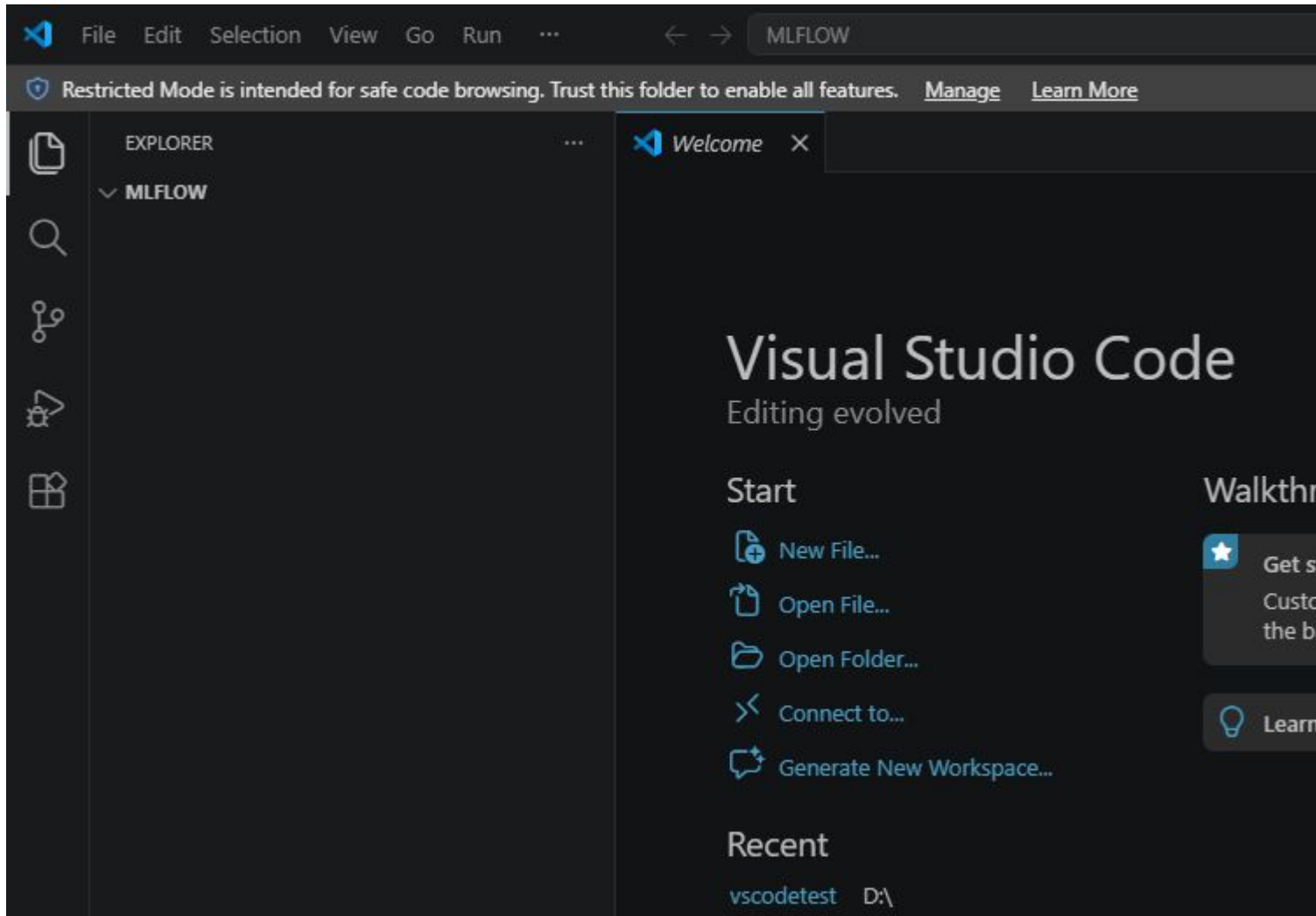
print("Run logged successfully!")
```

In viscode

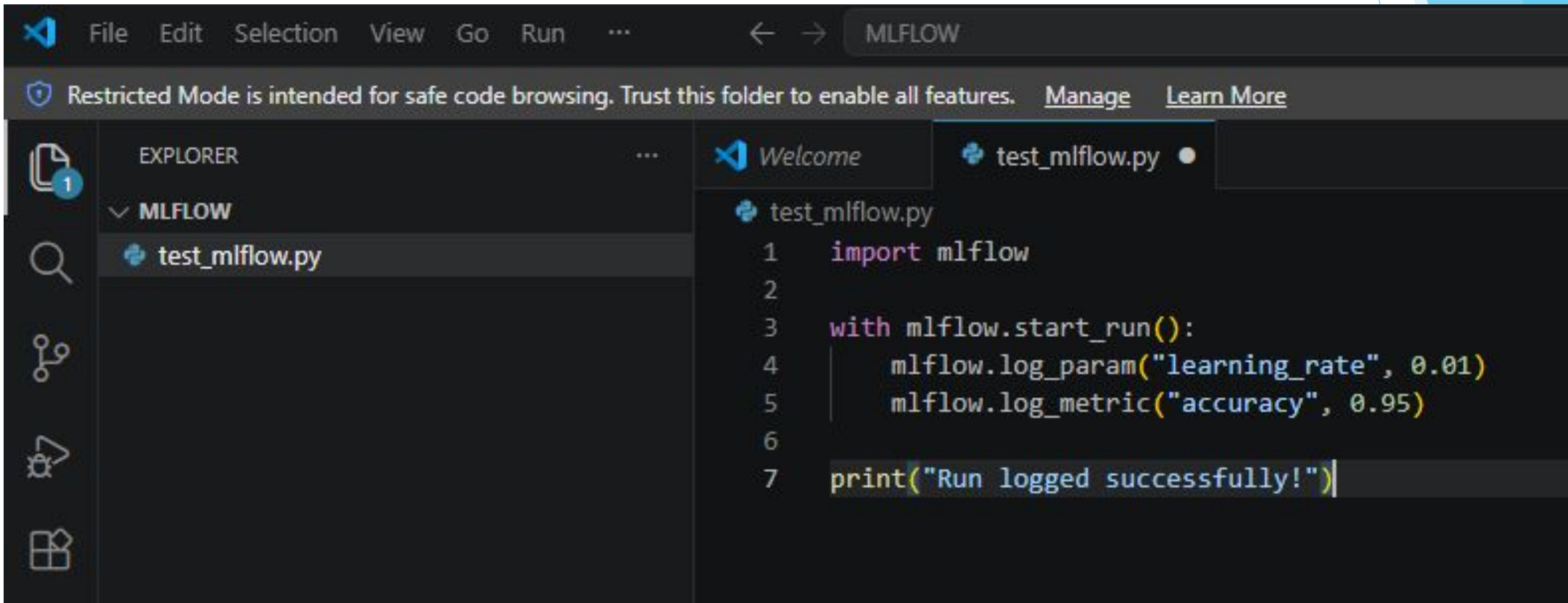
- Open any folder in viscode



Viscode main window



Create test_mlflow.py and type the following code



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a folder named 'MLFLOW' containing a file 'test_mlflow.py'. The main editor area displays the code for 'test_mlflow.py'.

```
File Edit Selection View Go Run ... MLFLOW
```

Restricted Mode is intended for safe code browsing. Trust this folder to enable all features. [Manage](#) [Learn More](#)

EXPLORER

MLFLOW

test_mlflow.py

test_mlflow.py

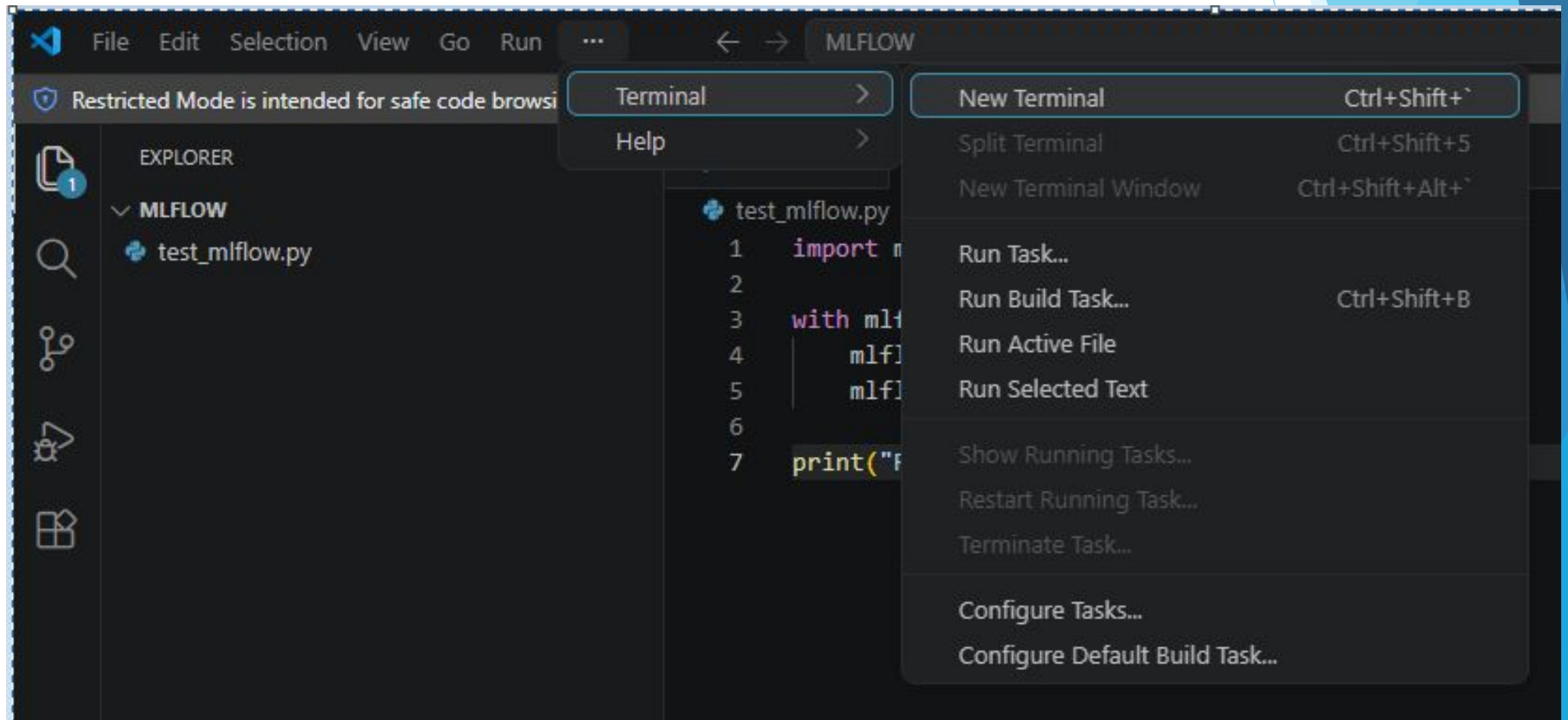
```
1 import mlflow
2
3 with mlflow.start_run():
4     mlflow.log_param("learning_rate", 0.01)
5     mlflow.log_metric("accuracy", 0.95)
6
7 print("Run logged successfully!")
```

Test code

- ▶ `import mlflow`
- ▶ `with mlflow.start_run():`
 - ▶ `mlflow.log_param("learning_rate", 0.01)`
 - ▶ `mlflow.log_metric("accuracy", 0.95)`
- ▶ `print("Run logged successfully!")`

Run this code using viscode terminal

- ▶ `python test_mlflow.py`



```
2  
3 with mlflow.start_run():  
4     mlflow.log_param("learning_rate", 0.01)  
5     mlflow.log_metric("accuracy", 0.95)  
6  
7 print("Run logged successfully!")
```

New Terminal	Ctrl+Shift+`
New Terminal Window	Ctrl+Shift+Alt+`
Split Terminal	Ctrl+Shift+5

PowerShell

Git Bash

Command Prompt

JavaScript Debug Terminal

Agent Host (Local)

Split Terminal with Profile

Configure Terminal Settings

Select Default Profile

Run Task...

Configure Tasks...

PROBLEMS

OUTPUT

TERMINAL

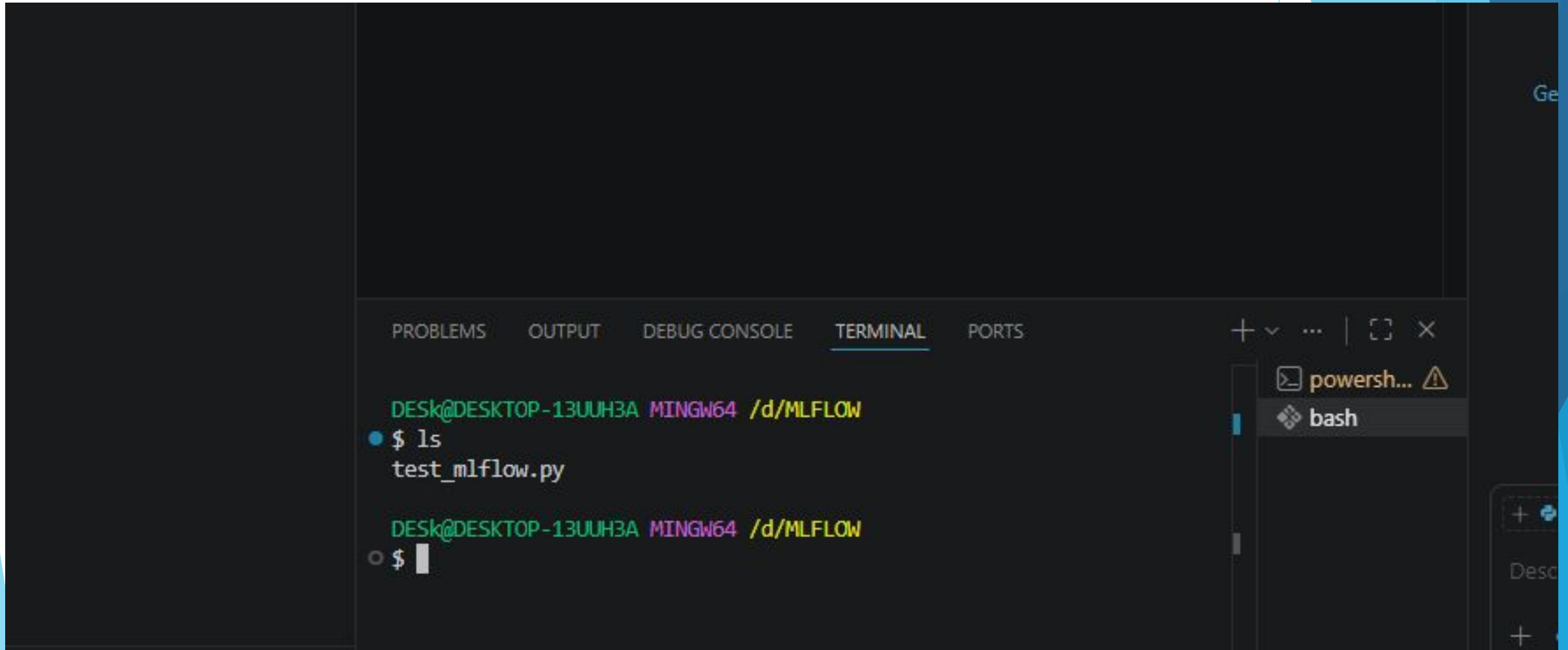
...

powershell



PS D:\MLFLOW>

Bring Git bash terminal in viscode



The screenshot shows the Visual Studio Code interface with the terminal panel open. The terminal shows a Git Bash session. The prompt is 'DESK@DESKTOP-13UJH3A MINGW64 /d/MLFLOW'. The user has entered the command 'ls' and the output is 'test_mlflow.py'. The terminal also shows a second prompt 'DESK@DESKTOP-13UJH3A MINGW64 /d/MLFLOW' with a dollar sign and a cursor. On the right side of the terminal panel, there is a dropdown menu with 'powersh...' and 'bash' options. The 'bash' option is selected.

```
DESK@DESKTOP-13UJH3A MINGW64 /d/MLFLOW
$ ls
test_mlflow.py

DESK@DESKTOP-13UJH3A MINGW64 /d/MLFLOW
$
```

Run python and mlflow ui



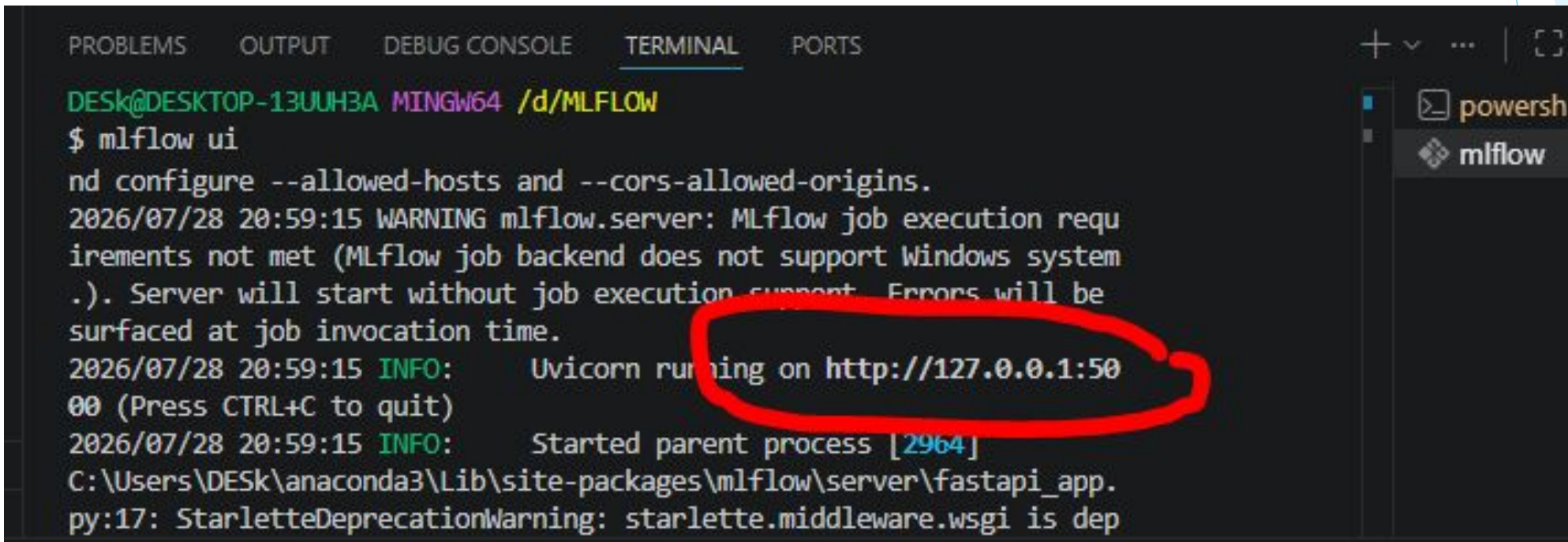
A screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The prompt is 'DESK@DESKTOP-13UUH3A MINGW64 /d/MLFLOW'. The first command is '\$ python test_mlflow.py'. The second command is '\$ ls', which outputs 'mlflow.db' and 'test_mlflow.py'. The third command is '\$ mlflow ui', which is currently being entered with a cursor at the end.

```
DESK@DESKTOP-13UUH3A MINGW64 /d/MLFLOW
• $ python test_mlflow.py

DESK@DESKTOP-13UUH3A MINGW64 /d/MLFLOW
• $ ls
mlflow.db  test_mlflow.py

DESK@DESKTOP-13UUH3A MINGW64 /d/MLFLOW
○ $ mlflow ui
█
```

Local ip address for mlflow ui



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
DESK@DESKTOP-13UJH3A MINGW64 /d/MLFLOW
$ mlflow ui
nd configure --allowed-hosts and --cors-allowed-origins.
2026/07/28 20:59:15 WARNING mlflow.server: MLflow job execution requirements not met (MLflow job backend does not support Windows system .). Server will start without job execution support. Errors will be surfaced at job invocation time.
2026/07/28 20:59:15 INFO:      Uvicorn running on http://127.0.0.1:5000 (Press CTRL+C to quit)
2026/07/28 20:59:15 INFO:      Started parent process [2964]
C:\Users\DESk\anaconda3\Lib\site-packages\mlflow\server\fastapi_app.py:17: StarletteDeprecationWarning: starlette.middleware.wsgi is deprecated
```

MLflow Tracking

- ▶ The purpose of this quickstart is to provide a quick guide to the most essential core APIs of MLflow Tracking. In just a few minutes of following along with this quickstart, you will learn:
- ▶ How to **log** parameters, metrics, and a model using the MLflow logging API
- ▶ How to navigate to a model in the **MLflow UI**
- ▶ How to **load** a logged model for inference

1. In python `pip install mlflow`

- ▶ `import mlflow`
- ▶ `mlflow.set_experiment("MLflow Quickstart")`

2. Prepare training data

- ▶ Before training our first model, let's prepare the training data and model hyperparameters.

```
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load the Iris dataset
X, y = datasets.load_iris(return_X_y=True)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the model hyperparameters
params = {
    "solver": "lbfgs",
    "max_iter": 1000,
    "random_state": 8888,
}
```

Step 3 - Train a model with MLflow Autologging

- ▶ In this step, we train the model on the training data loaded in the previous step, and log the model and its metadata to MLflow.
- ▶ The easiest way to do this is to using MLflow's **Autologging** feature.

- ▶ `import mlflow`
- ▶ `# Enable autologging for scikit-learn`
- ▶ `mlflow.sklearn.autolog()`
- ▶ `# Just train the model normally`
- ▶ `lr = LogisticRegression(**params)`
- ▶ `lr.fit(X_train, y_train)`

- ▶ With just one line of additional code `mlflow.sklearn.autolog()`, now you get the best of both worlds:
- ▶ you can focus on training the model, and MLflow will take care of the rest
 - ▶ Saving the trained model.
 - ▶ Recording the model's performance metrics during training, such as accuracy, precision, AUC curve.
 - ▶ Logging hyperparameter values used to train the model.
 - ▶ Track metadata such as input data format, user, timestamp, etc.

4 - View the Run in the MLflow UI

- ▶ To see the results of training, you can access the MLflow UI by navigating to the URL of the **Tracking Server**.
- ▶ If you have not started one, open a new terminal and run the following command at the root of the MLflow project and access the UI at
- ▶ <http://localhost:5000>
- ▶ (or the port number you specified).
- ▶ **mlflow server --port 5000**